

02 — pgvector & Dual-Index Schema Design

What it is

A **database schema** is the blueprint for how your data is organized — what tables exist, what columns they have, what types of data each column holds, and what indexes speed up searches. In this project, the schema is designed around a concept called **dual-path indexing**: every piece of text is stored in two searchable formats simultaneously:

1. **Vector embedding** (the "meaning" path) — a list of 768 numbers that captures *what the text means*. Similar texts end up with similar numbers, so you can find "companies that do something similar" even if they use completely different words.
2. **BM25 tsvector** (the "keyword" path) — a processed list of word roots (lemmas) that PostgreSQL can search very fast using its built-in full-text search. This is for "find companies that mention the exact word 'forklift'."

Real-life analogy

Imagine you run a huge job board. You need two ways to match candidates to jobs:

1. **The "vibe match"** — a senior recruiter reads every resume and writes a summary score: "This person is 87% similar to what Company X needs." That's the vector embedding — it captures meaning even when the exact words differ ("software engineer" matches "developer" matches "programmer").
2. **The keyword filter** — an automated system checks: "Does this resume literally contain the words 'Python', 'FastAPI', 'PostgreSQL'?" That's BM25 — fast, exact, no interpretation.

Neither alone is perfect. The vibe match might say a Java developer is "similar" to a Python role (they're both programming). The keyword filter misses resumes that say "Flask" instead of "FastAPI" even though they're closely related. **Using both together** (which we do with RRF fusion later) gives much better results.

Why we used it here

- **R9** requires dual-path indexing — every record needs both an embedding vector and lemmatized BM25 tokens.
- **R12** defines the specific schema: `category_vectors` and `company_vectors` tables.
- **R24** requires an `app_config` table so system prompts and provider settings are stored in the database (editable via Admin UI) rather than hardcoded.

The Technical Spec explicitly states: the embedding path uses the **raw** (un-lemmatized) narrative text, while the BM25 path uses **lemmatized** tokens. This is intentional — embeddings capture meaning from inflected forms (Hungarian "kínálunk" vs "kínál" carry slightly different nuances), while BM25 needs normalized root forms to match keywords reliably.

Files: `bilingual_etl/load/schema.sql`, `bilingual_etl/load/db.py`

Code walkthrough

```
-- The pgvector extension must be enabled once per database.
-- This gives PostgreSQL the VECTOR data type and similarity operators.
CREATE EXTENSION IF NOT EXISTS vector;

-- =====
-- category_vectors: one row per language per category
```

```

-- =====
CREATE TABLE IF NOT EXISTS category_vectors (
    id                BIGSERIAL PRIMARY KEY,
    -- BIGSERIAL = auto-incrementing integer, guaranteed unique

    category_id       TEXT NOT NULL,
    -- Links back to the source category from our JSON data

    language          TEXT NOT NULL CHECK (language IN ('hu', 'en')),
    -- CHECK constraint: database rejects any value other than 'hu' or 'en'.
    -- This is a safety net – code bugs can't silently insert 'hungarian'.

    narrative         TEXT NOT NULL,
    -- The full prose narrative text (used for the embedding path).
    -- NOT the lemmatized version – embeddings need the raw text.

    embedding         VECTOR(768),
    -- 768 numbers representing the "meaning" of the narrative.
    -- 768 is the dimension for Gemini embeddings at MRL size.
    -- pgvector stores this as a compact binary, not 768 separate columns.

    bm25_tokens_hu    TSVECTOR,
    bm25_tokens_en    TSVECTOR,
    -- TSVECTOR is PostgreSQL's built-in type for full-text search.
    -- Two separate columns because Hungarian and English need different
    -- dictionaries/stemmers – you can't mix them in one tsvector.

    metadata          JSONB NOT NULL DEFAULT '{}',
    -- Flexible JSON storage for AI enrichment output:
    -- ai_type, status, keywords, headline, synthetic_questions, etc.
    -- JSONB (not JSON) = stored in a binary format that's faster to query.

    content_hash      TEXT NOT NULL,
    -- MD5 hash of the source content. If this hasn't changed since last run,
    -- we skip re-translation/re-embedding (R10: hash-based change detection).

    updated_at        TIMESTAMPTZ NOT NULL DEFAULT now(),
    -- Timestamp with timezone. Automatically set on insert.

    UNIQUE (category_id, language)
    -- One HU row + one EN row per category. Trying to insert a second
    -- HU row for the same category_id will fail – enforces bilingual parity.
);

-- =====
-- Indexes: what they are and why each exists
-- =====

-- HNSW index on the embedding column.
-- HNSW = "Hierarchical Navigable Small World" – an algorithm that builds
-- a graph structure for fast approximate nearest-neighbor search.
-- vector_cosine_ops = use cosine similarity (not Euclidean distance).
-- Without this index, every search would scan ALL rows – O(n).
-- With it, search is O(log n) – fast even with millions of rows.
CREATE INDEX IF NOT EXISTS idx_category_vectors_embedding
    ON category_vectors USING hnsw (embedding vector_cosine_ops);

-- GIN index on tsvector columns.
-- GIN = "Generalized Inverted Index" – like the index at the back of a
-- textbook. Instead of scanning every row for a keyword, PostgreSQL looks
-- up the keyword in the index and jumps straight to matching rows.
CREATE INDEX IF NOT EXISTS idx_category_bm25_hu
    ON category_vectors USING gin (bm25_tokens_hu);

-- GIN on JSONB metadata – lets us filter by metadata fields efficiently.
-- Example: WHERE metadata->>'ai_type' = 'GÉP_ÉS_BERENDEZÉS'
CREATE INDEX IF NOT EXISTS idx_category_metadata
    ON category_vectors USING gin (metadata);

-- B-tree index on category_id – fast exact lookups.
-- B-tree is the "default" index type, good for = and range queries.

```

```

CREATE INDEX IF NOT EXISTS idx_category_id
    ON category_vectors (category_id);

# bilingual_etl/load/db.py – Database connection module

import os
from pathlib import Path
import psycopg2
from psycopg2.extras import RealDictCursor

# Read the connection string from an environment variable.
# Falls back to the local Docker Compose default if not set.
# Format: postgresql://user:password@host:port/database
DATABASE_URL = os.getenv(
    "DATABASE_URL",
    "postgresql://bilingual:bilingual@localhost:5432/bilingual_search",
)

# Path to the SQL schema file (relative to this Python file's location)
SCHEMA_PATH = Path(__file__).parent / "schema.sql"

def get_connection():
    # psycopg2.connect() opens a TCP connection to the PostgreSQL server.
    # The connection object lets you run SQL queries.
    return psycopg2.connect(DATABASE_URL)

def get_dict_cursor(conn):
    # Normal cursors return rows as tuples: (1, 'hello', 42)
    # RealDictCursor returns them as dicts: {'id': 1, 'name': 'hello', 'age': 42}
    # Much easier to work with in Python.
    return conn.cursor(cursor_factory=RealDictCursor)

def init_schema():
    # Read the entire SQL file as a string and execute it.
    # This creates all tables and indexes if they don't already exist.
    sql = SCHEMA_PATH.read_text()
    conn = get_connection()
    try:
        with conn.cursor() as cur:
            cur.execute(sql)
            conn.commit() # Make the changes permanent
    finally:
        conn.close() # Always close connections to avoid resource leaks

```

How to explain this in an interview / to a teammate

"Our schema implements dual-path indexing — every record is stored with both a vector embedding for semantic search and BM25 tsvector tokens for keyword search, in the same PostgreSQL database using the pgvector extension. We chose HNSW indexes for vector search because they give fast approximate nearest-neighbor results without a training step, and GIN indexes for the tsvector columns because they're PostgreSQL's standard way to speed up full-text search. Each record gets two rows — one Hungarian, one English — enforced by a UNIQUE constraint, so the database itself prevents us from accidentally breaking bilingual parity. We also have an `app_config` table for dynamic settings that the Admin UI can change without redeployment."